PROGRAM 12 12.c

Singly Linked List Operations

C SOURCE CODE 12.c

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
 int data;
 struct Node*next;
};
struct Node*head=NULL;

void insertAtEnd(int value) {
struct Node *newNode=(struct Node*) malloc(sizeof(struct Node));
newNode->data=value;
newNode->next=NULL;
if(head==NULL) {
 head= newNode;
 }
else {
 struct Node *temp=head;
 while (temp->next!=NULL) {
 temp=temp->next;
 }
temp->next=newNode;
}
}

void insertAtBeginning(int value) {
struct Node *newNode=(struct Node*) malloc(sizeof(struct Node));
newNode->data=value;
newNode->next=head;
head=newNode;
}

void insertAtPosition(int value,int position) {
struct Node *newNode=(struct Node*) malloc(sizeof(struct Node));
newNode->data=value;
newNode->next=NULL;
if (position==1) {
newNode->next=head;
head=newNode;
return;
}
struct Node *temp=head;
for(int i=1;i<position-1;i++) {
if(temp==NULL) {
 printf("invalid position:\n");
 return;
}
temp=temp->next;
temp->next=newNode;
}
}
void deleteAtBeginning() {
if (head==NULL) {
 printf("list is empty:\n");
 return;
}
struct Node *temp=head;
head=head->next;
free(temp);
}

void deleteAtEnd() {
if (head==NULL) {
 printf("list is empty:\n");
```

```
 return;
}
if (head→next==NULL){
 free(head);
 head=NULL;
 return;
}
struct Node *temp=head;
while(temp→next→next≠NULL){
temp=temp→next;
}
free(temp→next);
temp→next=NULL;
}

void deleteAtPosition() {
int position;
if (head==NULL) {
 printf("list is empty:\n");
 return;
 }
if (position==1) {
struct Node *temp=head;
head=head→next;
free(temp);
return;
}
struct Node *temp=head;
for(int i=1;i<position-1;i++) {
if(temp==NULL) {
 printf("invalid position:\n");
 return;
}
temp=temp→next;
}
if(temp→next==NULL) {
printf("invalid position! \n");
return;
}
struct Node *deleteNode=temp→next;
temp→next=deleteNode→next;
free(deleteNode);
}

void display() {
if (head==NULL) {
printf("list is empty! \n");
return;
}
struct Node *temp=head;
printf("linked list:");
while (temp≠NULL) {
printf("%d→",temp→data);
temp=temp→next;
}
printf("NULL\n");
}

int main() {
int choice,value,position;
while(1) {
printf("\n singly linked list operation   \n");
printf("\n1.insert at End\n");
printf("\n2.insert at Beginning\n");
printf("\n3.insert at Position\n");
printf("\n4.delete at Beginning\n");
printf("\n5.delete at End\n");
printf("\n6.delete at Position\n");
printf("\n7.display list\n");
printf("\n8.exit\n");
```

```c
printf("enter your choice:");
scanf("%d",&choice);
switch (choice) {
 case 1:
    printf("enter value to insert at End:");
    scanf("%d",&value);
    insertAtEnd(value);
    break;
 case 2:
    printf("enter value to insert at Beginning:");
    scanf("%d",&value);
    insertAtBeginning(value);
    break;
 case 3:
    printf("enter value to insert:");
    scanf("%d",&value);
    printf("enter position:");
    scanf("%d",&position);
    insertAtPosition(value,position);
    break;
 case 4:
    deleteAtBeginning();
    break;
 case 5:
    deleteAtEnd();
    break;
 case 6:
    printf("enter position to delete:");
    scanf("%d",&position);
    deleteAtPosition(position);
    break;
 case 7:
    display();
    break;
 case 8:
    printf("exiting  \n");
    exit(0);
    default:
    printf("invalid choice!try again\n");
  }
 }
return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
- - - Linked List Menu - - -
1. Create
2. Display
3. Search
4. Insert at Beginning
5. Insert at Middle
6. Insert at End
7. Delete at Beginning
8. Delete at Middle
9. Delete at End
10. Exit
Enter choice : 1
Enter data : 10
Node created successfully

- - - Linked List Menu - - -
Enter choice : 4
Enter data : 5
Node inserted successfully at beginning

- - - Linked List Menu - - -
Enter choice : 6
Enter data : 30
```

```
Node inserted successfully at end

- - - Linked List Menu - - -
Enter choice : 5
Enter data : 20
Enter position : 3
Node inserted successfully at position 3

- - - Linked List Menu - - -
Enter choice : 2
Linked List: 5 → 10 → 20 → 30 → NULL

- - - Linked List Menu - - -
Enter choice : 3
Enter element to search : 20
Element 20 found at position 3

- - - Linked List Menu - - -
Enter choice : 8
Enter position to delete : 3
Deleted element: 20

- - - Linked List Menu - - -
Enter choice : 7
Deleted element: 5

- - - Linked List Menu - - -
Enter choice : 9
Deleted element: 30

- - - Linked List Menu - - -
Enter choice : 2
Linked List: 10 → NULL

- - - Linked List Menu - - -
Enter choice : 10
Exiting ...
```

PROGRAM 13 13.c

Doubly Linked List Operations

C SOURCE CODE 13.c

```c
/* Implementation of Stack using Linked List in C */

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *top = NULL;

void push() {
    int val;
    printf("Enter value: ");
    scanf("%d", &val);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = top;

    if (top != NULL) {
        newNode->next= top;
    }

    top = newNode;
}

int pop() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return -1;
    }
    struct Node *temp = top;
    top = top->next;
    printf("Popped element: %d\n", temp->data);
    free(temp);
}

void display() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return;
    }
    printf("Displaying contents:\n");
    printf("NULL\n");
    struct Node *temp = top;
    while (temp != NULL) {

        printf("%d\n ", temp->data);
        temp = temp->next;
    }
}

void peek(){
    if(top == NULL) {
        printf("Stack Underflow\n");
        return;
    }
    printf("Displaying contents:\n");
    struct Node *temp = top;
    if (temp != NULL) {
        printf("%d\n", temp->data);
```

```c
    }
}


int main() {
    int choice, value;

    do {
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Display\n");
        printf("4. PEEK\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                push();
                break;
            case 2:
                pop();
                break;
            case 3:
                display();
                break;
            case 4:
                peek();
                break;
            case 5:
                printf("Exiting ... \n");
                break;
            default:
                printf("Invalid choice. Please try again.\n");
        }
    } while (choice ≠ 5);

    return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
- - - Doubly Linked List Menu - - -
1. Create
2. Display
3. Search
4. Insert at Beginning
5. Insert at Middle
6. Insert at End
7. Delete at Beginning
8. Delete at Middle
9. Delete at End
10. Exit
Enter choice : 1
Enter data : 10
Node created successfully

- - - Doubly Linked List Menu - - -
Enter choice : 4
Enter data : 5
Node inserted successfully at beginning

- - - Doubly Linked List Menu - - -
Enter choice : 6
Enter data : 30
Node inserted successfully at end

- - - Doubly Linked List Menu - - -
Enter choice : 5
```

```
Enter data : 20
Enter position : 3
Node inserted successfully at position 3

- - - Doubly Linked List Menu - - -
Enter choice : 2
Linked List: 5 ⟷ 10 ⟷ 20 ⟷ 30 ⟷ NULL

- - - Doubly Linked List Menu - - -
Enter choice : 3
Enter element to search : 20
Element 20 found at position 3

- - - Doubly Linked List Menu - - -
Enter choice : 8
Enter position to delete : 3
Deleted element: 20

- - - Doubly Linked List Menu - - -
Enter choice : 7
Deleted element: 5

- - - Doubly Linked List Menu - - -
Enter choice : 9
Deleted element: 30

- - - Doubly Linked List Menu - - -
Enter choice : 2
Linked List: 10 ⟷ NULL

- - - Doubly Linked List Menu - - -
Enter choice : 10
Exiting ...
```

PROGRAM 14 14.c

Stack Operations Using Linked List

C SOURCE CODE 14.c

```c
/* Implementation of Queue using Linked List in C */

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *front = NULL;
struct Node *rear = NULL;

void enqueue() {
    int val;
    printf("\n Enter value to enqueue: ");
    scanf("%d", &val);
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));

    newNode->data = val;
    newNode->next = NULL;

    if (front == NULL && rear == NULL) {
        front = rear = newNode;
    } else {
        rear->next = newNode;
        rear = newNode;
    }
}

void dequeue() {
    if (front == NULL && rear == NULL) {
        printf("Queue Underflow\n");
        return;
    }
    struct Node *temp = front;
    int val = temp->data;
    front = front->next;
    printf("Dequeued: %d\n", val);
    free(temp);
    if (front == NULL) rear = NULL;
}

void display() {
    if (front == NULL && rear == NULL) {
        printf("Queue Empty\n");
        return;
    }
    printf("Displaying Contents:\n");
    printf("NULL\t");

    struct Node *temp = front;
    while (temp != NULL) {
        printf("%d\t", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
void peek() {
    if (front == NULL && rear == NULL) {
        printf("Queue Empty\n");
        return;
    }
```

```c
    printf("Displaying last Contents:\n");
    struct Node *temp = front;
    if(temp != NULL) {
        printf("%d\n", temp->data);
    }
}

int main() {
    int choice, value;

    do {
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Display\n");
        printf("4. PEEK\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                enqueue();
                break;
            case 2:
                dequeue();
                break;
            case 3:
                display();
                break;
            case 4:
                peek();
                break;
            case 5:
                printf("Exiting ... \n");
                break;
            default:
                printf("Invalid choice! Please try again.\n");
        }
    } while (choice != 5);

    return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value to push: 10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value to push: 20
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Displaying contents:
20
10
1. Push
2. Pop
3. Display
```

```
4. Exit
Enter your choice: 2
Popped value: 20
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Displaying contents:
10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 4
Exiting ...
```

PROGRAM 15 15.c

Queue Operations Using Linked List

C SOURCE CODE 15.c

```c
#include<stdio.h>
#include<stdlib.h>
void main()
{
    int arr[100],key=0,found,i,n;
    int t;
    printf("enter no of elements");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
    printf("enter elements");
    scanf("%d",&arr[i]);
    }
    // display the elements
    printf("DISPLAY \n");
    for(i=0;i<n;i++)
    {
    printf("%d element :%d\t",i,arr[i]);
    printf("\t");
    }
    printf("\n");

    printf("enter the element to be searched");
    scanf("%d",&found);
    for(i=0;i<n;i++)
    {
        if(found==arr[i])
        {
            key=1;
            t=i;
        }
    }
    if(key==0)
    {
        printf("element is not on the array");

    }
    else
    {
        printf("element found at%d",t);
    }

}
```

PROGRAM OUTPUT Execution Verification

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 100
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 200
1. Enqueue
2. Dequeue
3. Display
4. Exit
```

```
Enter your choice: 3
Displaying Contents:
100
200
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Dequeued value: 100
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Displaying Contents:
200
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
Exiting ...
```